

These test were done on 750,000 records due to computational and budget limitations.

Author Search

The screenshot displays the Postman interface for a REST client. The workspace is named 'My Workspace' and contains a collection 'Hybrid Search API'. The selected environment is 'No environment'. The active request is a POST to 'http://localhost:8000/search' with a body of:

```
1 {
2   "query": "Julia Alvarez",
3   "top_k": 2,
4   "from_": 0
5 }
```

The response is a 200 OK status with a 4 ms response time and 776 B of data. The response body is shown in JSON format:

```
1 [
2   {
3     "id": 200,
4     "title": "Country article political set former.",
5     "author": "<em>Julia</em> <em>Alvarez</em>"
  }
]
```

The console at the bottom shows the request and response details:

```
POST http://localhost:8000/search%0A 200 | 126 ms
POST http://localhost:8000/search%0A 200 | 4 ms
```

Result was correctly returned. First request took 126 ms. The result was cached. When tried again, the result is retrieved from cache hence response time reduces by ~ 30x.

Title Search

For this request, the cached result is returned with a ~ 90x reduction in response time

Hybrid Search API / Search for Author

POST http://localhost:8000/search

Params Authorization Headers (10) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "query": "Dog professor act I drop clear.",
3   "top_k": 15,
4   "from_": 0
5 }
```

Body Cookies Headers (4) Test Results

200 OK • 8 ms • 4.45 KB • Save Response

Pretty Raw Preview Visualize JSON

```
1 [
2   {
3     "id": 19,
4     "title": "<em>Dog</em> <em>professor</em> <em>act</em> <em>I</em> <em>drop</em> <em>clear</em>.",
5     "author": "Kristin Alvarez",
```

Find and replace Console

All Logs Clear

http://localhost:8000/search%0A 200 | 725 ms

http://localhost:8000/search%0A 200 | 8 ms

Content Search

The mock data for magazine articles generated 500-1000 sentences for each magazine article record using the faker library. These were then handled in the 2 indices in elasticsearch, with the embeddings in magazine_content. Sentence and chunked information was also generated but these were disabled for single node clusters due to computational requirements

The hybrid search system implements a key feature from the coding challenge requirements: "Search based on vector similarity in the vector_representation field." This vector search uses cosine similarity to measure proximity between query and candidate vectors,

identifying contextually aligned results beyond simple keyword matches. By converting textual information into semantic vectors, this approach efficiently navigates vast datasets and captures nuanced relationships between data points.

While this vector search provides powerful contextual matching, it does not include additional semantic search capabilities (like sparse encodings or AI integrations like ELSER or GPT-4) for interpreting user intent. The current implementation focuses on vector similarity as specified, delivering efficient and contextually relevant results without the added layer of intent understanding that full semantic search would offer. Future work could involve extending this implementation to include them, as they are also computationally expensive and may require GPUs or multimode clusters.

a. Search by Phrase

The screenshot displays a REST client interface with the following details:

- Request:** POST to `http://localhost:8000/search`. The body is a JSON object: `{\"query\": \"Wind pass yeah reflect two\", \"top_k\": 100, \"from_\": 0}`.
- Response:** Status `200 OK`, `12 ms`, `29.15 KB`. The response body is a JSON object: `{\"id\": 19, \"title\": \"Dog professor act I drop clear.\", \"author\": \"Kristin Alvarez\", \"content\": \"<em>Wind</em> <em>pass</em> <em>yeah</em> <em>reflect</em> <em>two</em> through drop. Wear your my throughout. Difficult raise dog anything owner car.\"}`.
- Console:** Two log entries are visible: `:tp://localhost:8000/search%0A 200 | 1936 ms` and `:tp://localhost:8000/search%0A 200 | 12 ms`.

Query took more time due to the number of results required to be loaded (100). Caching reduces this long-term. Also, this phrase from the previous author search query is matched exactly but is not the top result. As discussed, the vector search does not factor in the intent of the user for full semantic search using ELSER or LLM (GPT-4).

b. Contextual Search

Hybrid Search API / Contextual search

POST http://localhost:8000/search

Params Authorization Headers (10) Body Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

```
1 {
2   "query": "Today is a good day",
3   "top_k": 10,
4   "from_": 0
5 }
```

Body Cookies Headers (4) Test Results

200 OK • 4 ms • 3.05 KB

Pretty Raw Preview Visualize JSON

```
4   "title": "Seven wrong million recognize north.",
5   "author": "Robert Martinez",
6   "content": "Quickly face determine year <em>good</em> one. Medical arrive evening build. Me happen season image clear couple. Cost environmental I
7   <em>a</em> but hundred would.",
8   "score": 0.6732239952,
```

Find and replace Console

All Logs Clear

http://localhost:8000/search%0A	200	199 ms
http://localhost:8000/search%0A	200	8 ms
http://localhost:8000/search%0A	200	4 ms

This search query expresses the sentiment of a good day and the top result has a positive sentiment associated with it. The least relevant result has no contextual similarity to the search query but just the keyword, highlighting the relevance of vector search in the hybrid search implementation

```
{  
  "id": 45,  
  "title": "City measure answer visit make.",  
  "author": "Erica Cardenas",  
  "content": "Education <em>day</em> approach stay own. Particularly site bag ability write side power. Such cold mention ask. Senior receive minute  
    <em>a</em>.",  
  "score": 0.6027678144,  
  "category": "son",  
}
```

With semantic search using GPT or ELSER, the distinction will be more pronounced.